

☐Hide inserted text ☐Hide deleted text

P0507R0: Core Issue 1343: Sequencing of non-class initialization

This paper presents the changes for core issue 1343. They are presented as a separate paper due to the volume of the changes.

Replace 1.9 [intro.execution] paragraph 10 by the following text, split into separate paragraphs at the editor's discretion:

A constituent expression is defined as follows:

- The constituent expression of an expression is that expression.
- The constituent expressions of a *braced-init-list* or of a (possibly parenthesized) *expression-list* are the constituent expressions of the elements of the respective list.
- The constituent expressions of a *brace-or-equal-initializer* of the form *= initializer-clause* are the constituent expressions of the *initializer-clause*.

[Example:

```
struct A { int x; };
struct B { int y; struct A a; };
B b = { 5, { 1+1 } };
```

The constituent expressions of the *initializer* used for the initialization of *b* are 5 and 1+1. -- end example]

The *immediate subexpressions* of an expression *e* are

- the constituent expressions of *e*'s operands (Clause 5),
- any function call that *e* implicitly invokes,
- if *e* is a *lambda-expression* (5.1.5 [expr.prim.lambda]), the initialization of the entities captured by copy and the constituent expressions of the *initializer* of the *init-captures*,
- if *e* is a function call (5.2.2 [expr.call]) or implicitly invokes a function, the constituent expressions of each default argument (8.3.6 [dcl.fct.default]) used in the call, or
- if *e* creates an aggregate object (8.5.1 [dcl.init.aggr]), the constituent expressions of each default member initializer (9.2 [class.mem]) used in the initialization.

A *subexpression* of an expression *e* is an immediate subexpression of *e* or a subexpression of an immediate subexpression of *e*. [Note: Expressions appearing in the *compound-statement* of a *lambda-expression* are not subexpressions of the *lambda-expression*.]

A *full-expression* is

- an unevaluated operand (Clause 5 [expr]),
- a *constant-expression* (5.20 [expr.const]),
- an *init-declarator* (Clause 8 [dcl.decl]) or a *mem-initializer* (12.6.2 [class.base.init]), including the constituent expressions of the *initializer*,
- an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (12.2 [class.temporary]), or
- an expression that is not a subexpression of another expression **and that is not otherwise part of a full-expression.**

[Note: in some contexts, such as unevaluated operands, a syntactic subexpression is considered a full-expression (Clause 5). -- end note] If a language construct is defined to produce an implicit call of a function, a use of the language construct is considered to be an expression for the purposes of this definition. **A call to a destructor generated at the end of the lifetime of an object other than a temporary object is an implicit full-expression.**

Conversions applied to the result of an expression in order to satisfy the requirements of the language construct in which the expression appears are also considered to be part of the full-expression. **For an initializer, performing the initialization of the entity (including evaluating default member initializers of an aggregate) is also considered part of the full-expression.** [Example:

```

struct S {
    S(int i): I(i) { } // full-expression is initialization of I
    int& v() { return I; }
    ~S() noexcept(false) { }
private:
    int I;
};
S s1(1); // full-expression is call of S::S(int)
void f() {
    S s2 = 2; // full-expression is call of S::S(int)
    void f() {
        if (S(3).v()) // full-expression includes lvalue-to-rvalue and
                        // int to bool conversions, performed before
                        // temporary is deleted at end of full-expression
        { }
        bool b = noexcept(S()); // exception specification of destructor of S considered for noexcept

        // full-expression is destruction of s2 at end of block
    }
    struct B {
        B(S = S(0));
    };
    B b[2] = { B(), B() }; // full-expression is the entire initialization including the destruction of temporaries
}
-- end example ]

```

Change in 5 [expr] paragraph 8:

In some contexts, unevaluated operands appear (5.2.8, 5.3.3, 5.3.7, 7.1.6.2). An unevaluated operand is not evaluated. ~~An unevaluated operand is considered a full-expression.~~ [Note: In an unevaluated operand, a non-static class member may be named (5.1) and naming of objects or functions does not, by itself, require that a definition be provided (3.2 [basic.def.odr]). **An unevaluated operand is considered a full-expression (1.9 [intro.execution]).** -- end note]

Change in 7.1.5 [dcl.constexpr] paragraph 9:

A constexpr specifier used in an object declaration declares the object as `const`. Such an object shall have literal type and shall be initialized. **In any constexpr variable declaration, the full-expression of the initialization shall be a constant expression (5.20 [expr.const]).** ~~If it is initialized by a constructor call, that call shall be a constant expression (5.20 [expr.const]). Otherwise, or if a constexpr specifier is used in a reference declaration, every full-expression that appears in its initializer shall be a constant expression. [Note: Each implicit conversion used in converting the initializer expressions and each constructor call used for the initialization is part of such a full-expression. -- end note]~~ [Example: ...]

Change in 12.6.2 [class.base.init] paragraph 7:

[Example: ...] **[Note:** The initialization performed by each *mem-initializer* constitutes a full-expression **(1.9 [intro.execution])**. Any expression in a *mem-initializer* is evaluated as part of the full-expression that performs the initialization. **-- end note]** A *mem-initializer* where the *mem-initializer-id* denotes a virtual base class is ignored during execution of a constructor of any class that is not the most derived class.

Change in 15.4 [except.spec] paragraph 12:

~~[...] A subexpression e1 of an expression e is an *immediate subexpression* if there is no subexpression e2 of e such that e1 is a subexpression of e2.~~